

Introduction to Algorithm Engineering

Spring 2026

Lecture 1

Agenda for Today

- Quick Welcome
- My details
 - Kishore Kothapalli, Professor, IIIT Hyderabad
 - Office: Vindhya, A3-102
 - Email: kkishore@iiit.ac.in (the best way to reach me)
 - Research interests span parallel computing and distributed algorithms.

Agenda for Today

- Rest of Today's Class
 - Syllabus
 - Policies
 - Expectations
 - Actual lecture

Syllabus

- Roughly, we will cover the following topics:
 - Basics
 - Cache-aware and cache-oblivious algorithms
 - Graph connectivity and decompositions
 - Fixed Parametrized Algorithms
 - Shortest Paths
 - Travelling Salesman Problem
 - Motif Counting

Evaluation

- Grading (Tentative)
 - Homeworks and In-class Worksheets: 25%
 - Course Project: 25%
 - End Exam: 30%
 - Quiz : 20%
 - Exceptional Performance: 5% extra, Exceptional performance in any component gets extra score.
- Any submission that is graded and evaluated **should not** be copied from any source.
- Copied submissions will get zero for the first instance and negative or more serious penalty (including F grade) for repeat offences.
- Penalty even for the first instance depends on the severity of the offence.

Textbooks

- Textbooks: Do not own these books just for the class!
 - Introduction to Algorithms, Leiserson et al.
 - Randomized Algorithms, Motwani and Raghavan
 - Introduction to Parallel Algorithms, J. JaJa
 - Other material to be posted on the course website
- My own book available at <http://faculty.iit.ac.in/~kkishore/book.pdf>
- Most welcome to write to me if you have any questions.

Expectations

- Utilize class time effectively.
 - Starts with all of you settling in the classroom by the start time of 830 AM.
 - Ask any question you may have. No question is small to ask.
 - Do not show up late. Lateness affects the entire class.

Agenda for Today

- On to the actual lecture....
- We will see how algorithm engineering can help in designing and analyzing algorithms.
- Starting with a very simple example...

Algorithm Engineering

- The (typical) algorithm design process
 - Design
 - Correctness
 - Theoretical Analysis

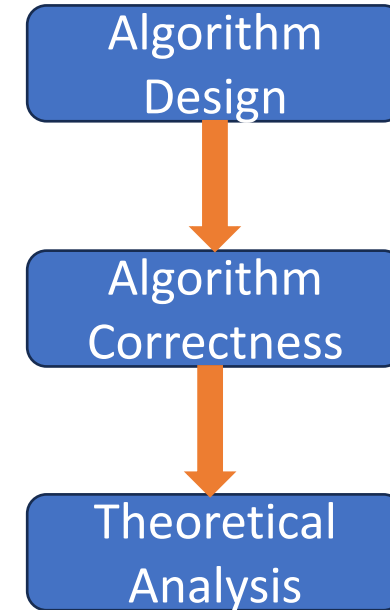


Fig 1: Algorithm Engineering

Picture inspired by G. Italiano, 2010

Algorithm Engineering

- The (typical) algorithm design process
 - Design
 - Correctness
 - Theoretical Analysis
 - Implementation
 - Experimental Studies

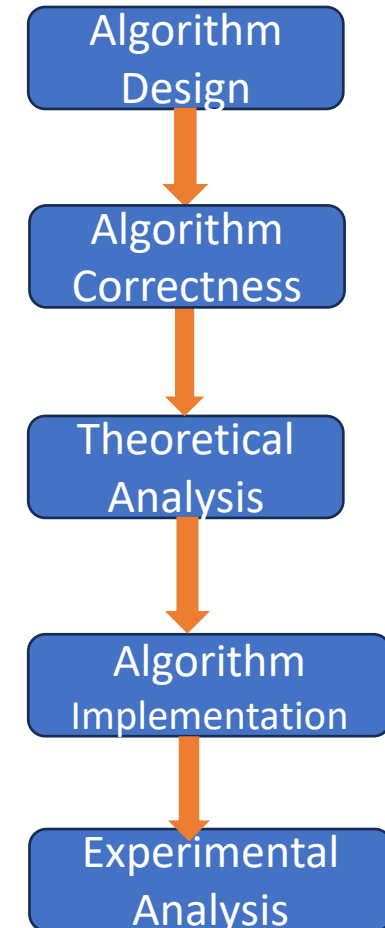


Fig 1: Algorithm Engineering

Picture inspired by G. Italiano, 2010

Algorithm Engineering

- The (typical) algorithm design process
 - Design
 - Correctness
 - Theoretical Analysis
 - Implementation
 - Experimental Studies
- We are aware of the above steps
 - Designed multiple algorithms
 - Argued about their correctness
 - Understood their asymptotic run time behavior
 - Implemented some of these algorithms

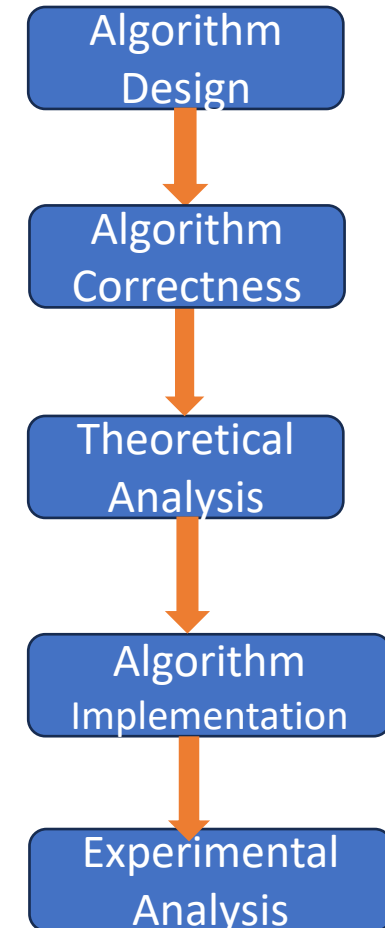


Fig 1: Algorithm Engineering

Algorithm Engineering

- A discipline closely aligned with algorithm design
- This discipline weaves the theoretical aspects of algorithms and their practical implementation related aspects.
- Offers interesting insights into the working details of algorithms while not compromising on correctness.

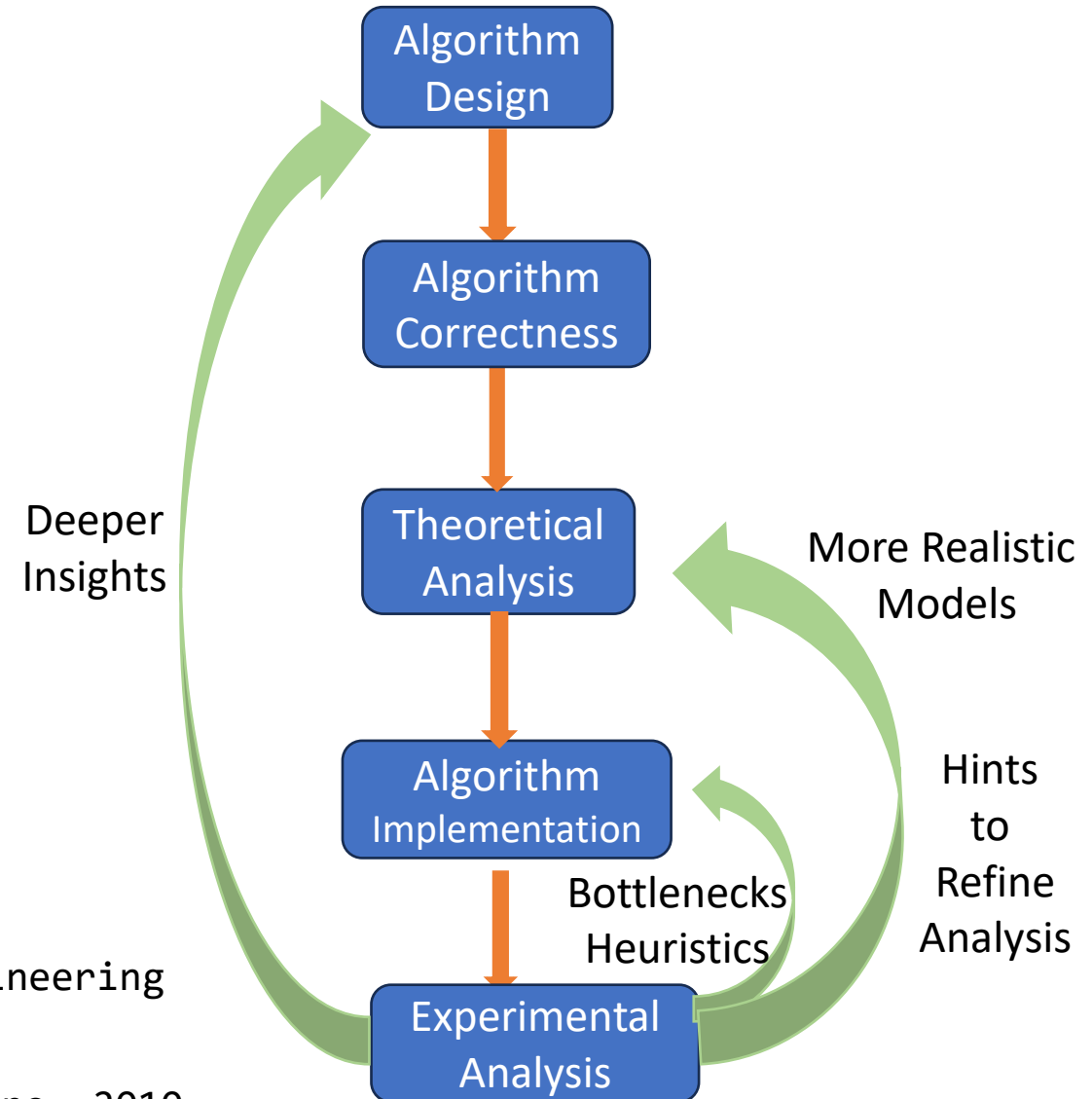


Fig 1: Algorithm Engineering

Picture credits to G. Italiano, 2010

Algorithm Engineering

- Complete the loop from algorithms to experiments to heuristics to algorithms
- Practitioners get to take a deep dive into in both algorithms and systems!

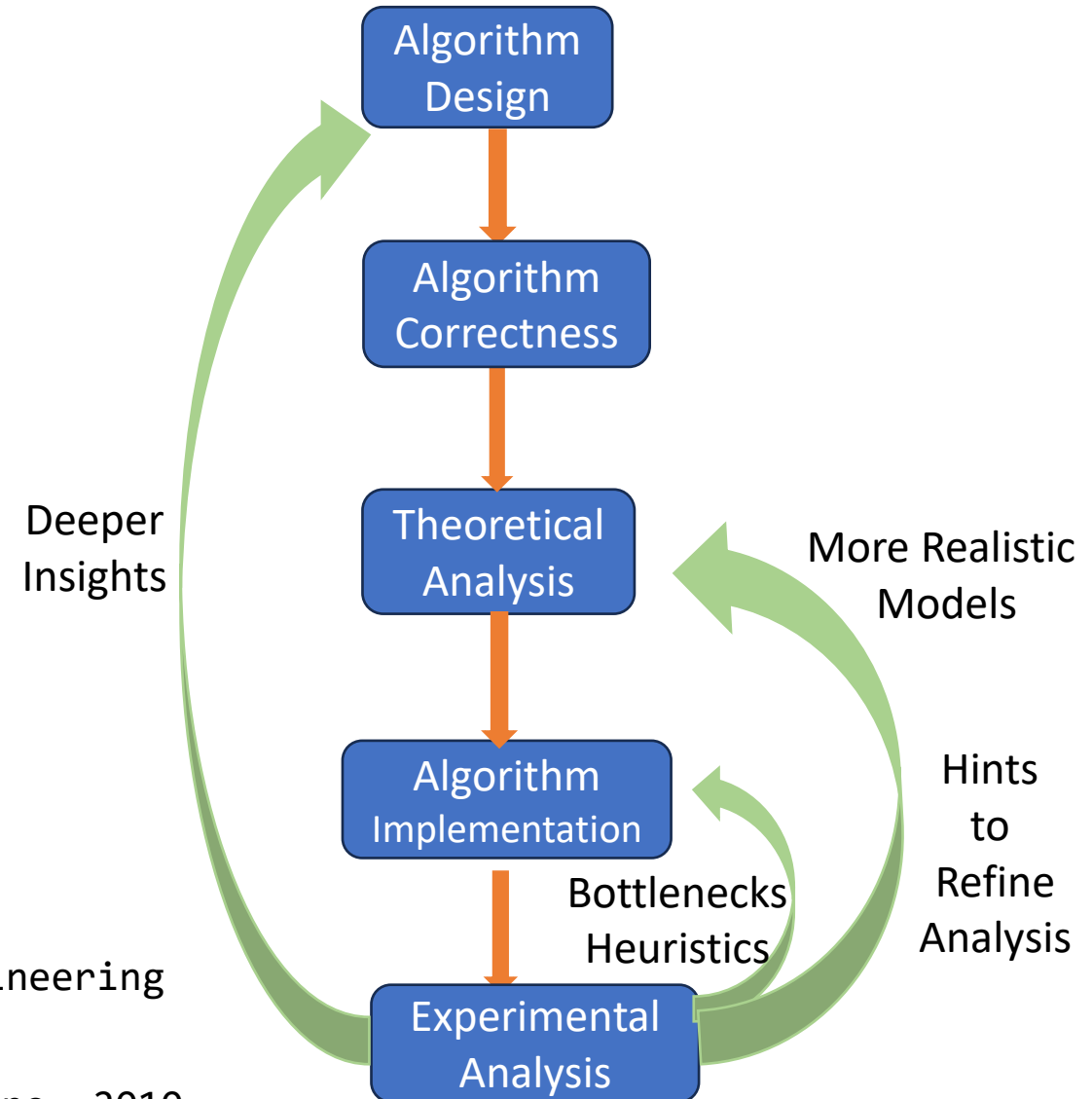


Fig 1: Algorithm Engineering

Picture credits to G. Italiano, 2010

An Example

- Consider any popular algorithm such as the insertion sort.
- We know that it has a worst case run time of $O(n^2)$.
- Consider some practical settings where sorting plays a role.
 - Library racks – they keep gaps on purpose.
- This observation was the key behind the design of a sorting algorithms called “library sort”.

An Example

- This observation was the key behind the design of a sorting algorithms called “library sort”.
- Key idea in library sort is to keep an array of size $\sim 2n$ with gaps in between.
- Invariant: Every block of $(2+\varepsilon)c \log n$ elements, there are at least $c \log n$ support elements. Hence, at most $(1+\varepsilon)c \log n$ elements inserted in this stretch.
- Reading exercise: Read about library sort from the paper posted in the course website (courses.iiit.ac.in)
- The paper shows that library sort, that is more like insertion sort with gaps, runs in $O(n \log n)$ time with high probability.

A Complete Example

- Let $G = (V, E)$ be an undirected graph on n vertices and m edges.
- The diameter of G is $\max_{v,w} d(v,w)$ where $d(v,w)$ is the shortest path distance between v and w .
 - The longest of the shortest distances.
 - Denoted $\text{diam}(G)$.

Diameter of a Graph

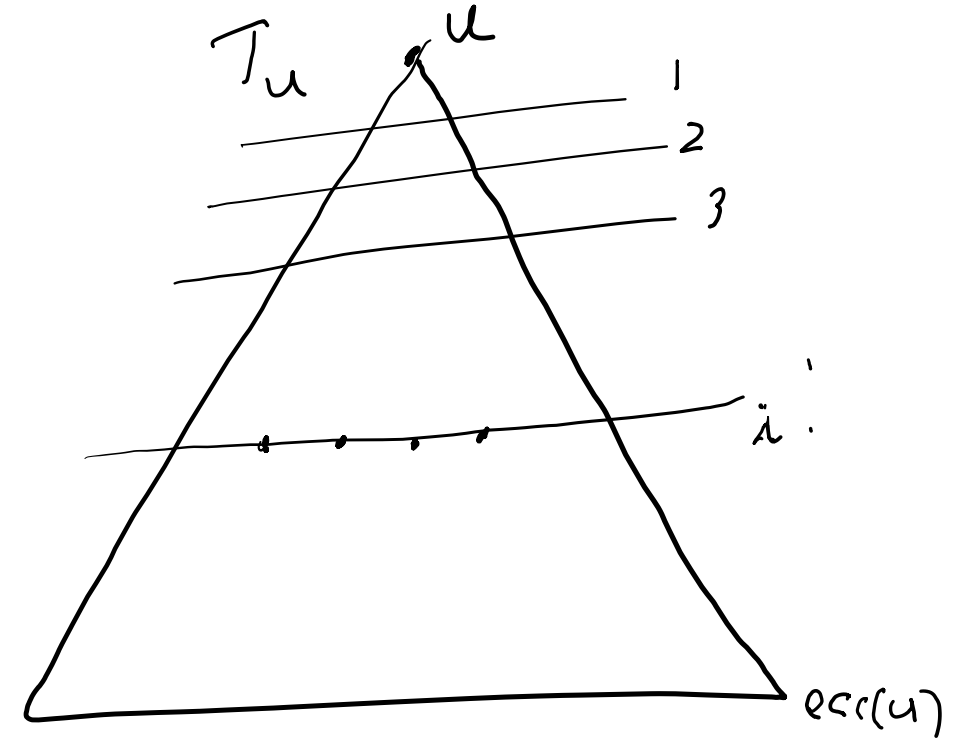
- The classical approach would be to use BFS from each vertex.
- The BFS from vertex v can be used find the farthest distance to some vertex w from v .
 - This quantity is also called as the **eccentricity** of v , denoted $\text{ecc}(v)$.
- The diameter of G is the maximum over all v , **$\text{ecc}(v)$** .
- This algorithm has a run time of $O(n(n+m))$ since each BFS runs in time $O(n+m)$.
- So far, there is no known algorithm that has a better run time.

Diameter of a Graph

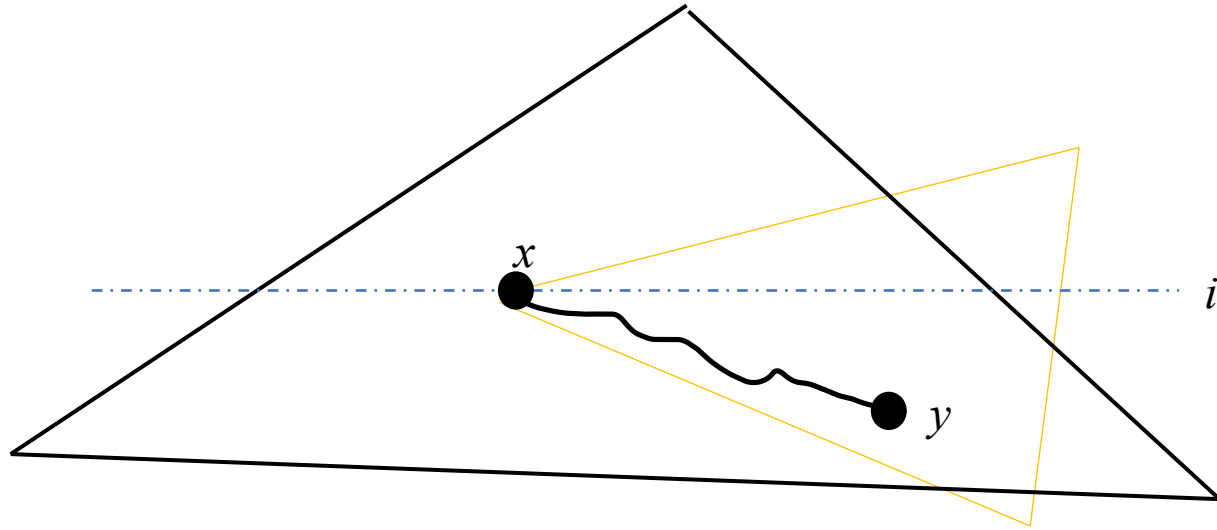
- It appears that there may be no better algorithm.
- However, a key question is to see if all the BFS are really required.
- A key observation by Crescenzi et al. helps improve the algorithm.

Diameter of a Graph

- Let us do a BFS from a node u in G .
- Let T_u be the BFS tree obtained.
- T_u has nodes in levels 0, 1, 2, etc. with node u in level 0, the neighbors of u in level 1, and the farthest nodes from u in level $\text{ecc}(u)$.
- Let $F_i(u)$ be the nodes at distance i from u .
 - F corresponds to **fringe**.
- Let $B_i(u) = \max_{z \in F_i(u)} \text{ecc}(z)$
- Note: $\text{diam}(G)$ is at most $2\text{ecc}(u)$. **Why?**
- We now show the following claims.



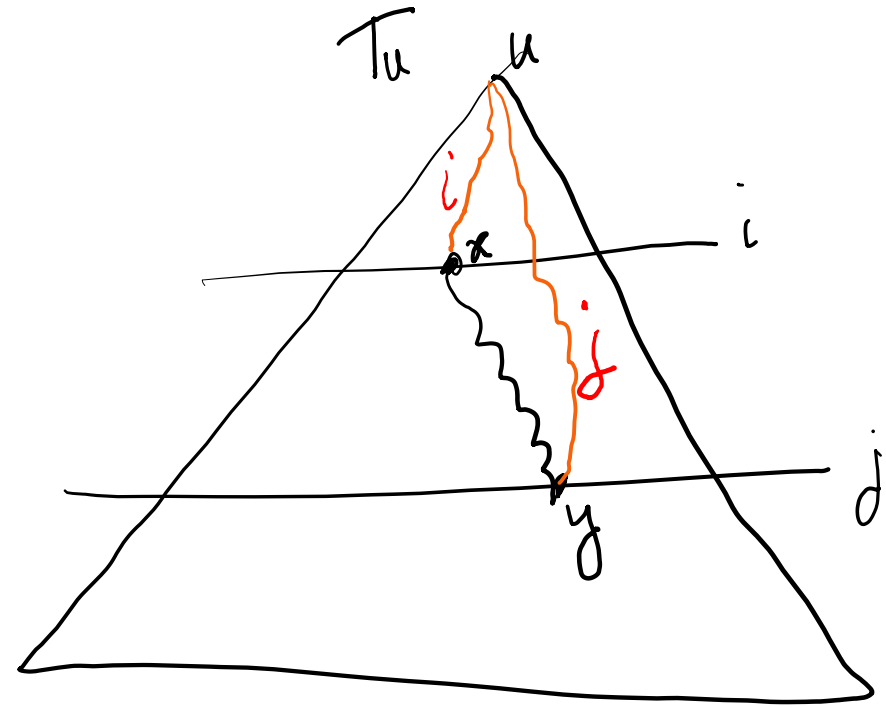
Diameter of a Graph



- Claim 1: Let x and y be any two nodes in G with at least one of them in $F_i(u)$. Then, $d(x, y) \leq B_i(u)$.
- Proof: Notice that $d(x, y) \leq \min \{ecc(x), ecc(y)\}$ by definition of $ecc(.)$.
- Suppose that x is in $F_i(u)$. Then, $ecc(x) \leq B_i(u)$. Same holds if y is also in $F_i(u)$ ending the proof.
- If y is not in $F_i(u)$, then x must be in $F_i(u)$ allowing us to conclude that $d(x, y) \leq ecc(x) \leq B_i(u)$.

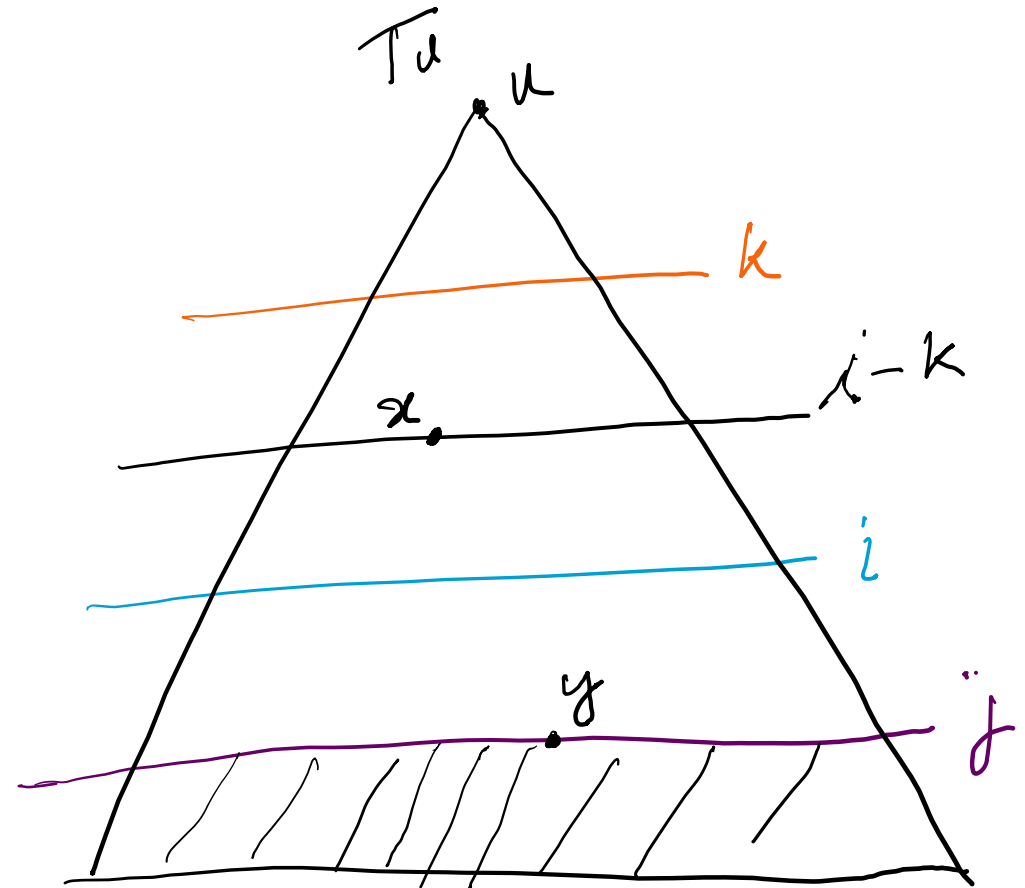
Diameter of a Graph

- Claim 2: Let x be in $F_i(u)$ and y be in $F_j(u)$. Then, $d(x, y) \leq i + j \leq 2 \max\{i, j\}$.
- Proof: Use the triangle inequality of distances.
- We note that $d(x, y) \leq d(x, u) + d(u, y) = i + j \leq 2 \max\{i, j\}$.



Diameter of a Graph

- Theorem 1: For any $1 \leq i < \text{ecc}(u)$ and $1 \leq k < i$, and for any $x \in F_{i-k}(u)$ such that $\text{ecc}(x) > 2(i-1)$, there exists $y_x \in F_j(u)$ such that $d(x, y_x) = \text{ecc}(x)$ with $j \geq i$.
- Proof: Given an x , there always exists a y_x such that $d(x, y_x) = \text{ecc}(x) > 2(i-1)$ by the hypothesis.
- Suppose that y_x is in $F_j(u)$ with $j < i$.
- Then, using Claim 2, $d(x, y_x) \leq 2(i-1)$, which is a contradiction.
 - In this case, $\max\{i, j\} = i$.
- Hence, y_x in $F_j(u)$ with $j \geq i$.



Diameter of a Graph

- We can reinterpret Theorem 1 as follows.
- If x is a node in $F_i(u) \cup F_{i+1}(u) \cup \dots \cup F_{\text{ecc}(u)}(u)$ with maximum eccentricity $\text{ecc}(x) > 2(i - 1)$, then the eccentricity of all nodes in $F_1(u) \cup F_2(u) \cup \dots \cup F_{i-1}(u)$ is not greater than $\text{ecc}(x)$.
- An algorithm can be derived from the above interpretation.
 - Pick a node u in G .
 - Perform a BFS in G from u and call the BFS tree as T_u .
 - Traverse T_u in a bottom-up fashion, starting from the nodes in $i = F_{\text{ecc}}(u)$.
 - At each level i , compute the eccentricities of all the nodes at level i .
 - If the maximum eccentricity e is greater than $2(i - 1)$ then we can discard traversing the remaining levels above level i .
 - The eccentricities of all such nodes cannot be greater than e .

Diameter of a Graph

ALGORITHM 1: iFUB

Input: A graph G , a node u , a lower bound ℓ for the diameter, and an integer k

Output: A value M such that $D - M \leq k$

$i := \text{ecc}(u);$

$lb := \max\{\text{ecc}(u), \ell\}; ub := 2\text{ecc}(u);$

while $ub - lb > k$ do

 if $\max\{lb, B_i(u)\} > 2(i - 1)$ then return $\max\{lb, B_i(u)\};$

 else $lb := \max\{lb, B_i(u)\}; ub := 2(i - 1);$

 end

$i := i - 1;$

end-while

return $lb;$

Diameter of a Graph

- What is the run time of the algorithm?
- In the worst case, it is possible that the while loop will go till $i = 0$.
- In that case, one has to do all n BFS's.
- So, the worst case run time is $O(n.m)$.
- What did we achieve then?

Diameter of a Graph

- In practice, the algorithm terminates much quickly.
- The authors study close to 200 graphs of a very large size and show that for several of them, only $0.1n$ BFS traversals are issued.
- A big improvement over the worst case run time.

Results

(a) Results obtained by ten executions of *iFUB*+4-SWEEPr, i.e. *iFUB* by using 4-SWEEPr.

Networks	Total	Number of networks having performance ratio v/n			
		$v/n \leq 0.001$	$0.001 < v/n \leq 0.01$	$0.01 < v/n \leq 0.1$	$0.1 < v/n \leq 1$
Autonomous systems graphs	2	2	0	0	0
Biological networks	48	4	22	22	0
Citations networks	5	4	0	1	0
Collaboration networks	13	4	7	2	0
Communication networks	38	26	7	4	1
Internet peer-to-peer networks	1	0	0	0	1
Meshes and electronic circuits	34	5	8	8	13
Product co-purchasing networks	4	4	0	0	0
Road networks	3	1	0	2	0
Social networks	11	9	0	2	0
Synthetic graphs	18	3	5	5	5
Web graphs	9	8	1	0	0
Words adjacency networks	4	1	3	0	0
Others	6	2	1	1	2
Total	196	73	54	47	22

Diameter of a Graph

- How to choose u ?
- Several ways:
 - Random choice made uniformly across the n vertices.
 - 4-Sweep: Use four BFSs as follows.
 - Let r_1 be a node in V .
 - Let a_1 be one of the farthest nodes from r_1
 - Let b_1 be one of the farthest nodes from a_1 .
 - r_2 is the node halfway between a_1 and b_1 ,
 - a_2 be the node farthest from r_2 , and b_2 = be the node farthest from a_2 .
 - Node u is then defined as the middle node of the path between a_2 and b_2 .
- Choose r_1 uniformly at random or as a node with the highest degree.

Diameter of the Graph

- This algorithm of Crescenzi et al. from the Theoretical Computer Science Journal in 2012 is a good example of algorithm engineering.
- We observe that theoretical claims lead to better algorithms but not in all cases.
- The paper has a couple of examples of graphs on which the proposed algorithm reaches a worst case time of $O(n.m)$.

Diameter of the Graph

- For instance, let the graph be a cycle on n nodes with n being odd.
- The diameter of the graph is $(n-1)/2$.
- From any node, the BFS branches out from the root into two branches.
- Each level of the BFS has two nodes. The eccentricity of each node is also $(n-1)/2$.
- The algorithm can stop only when $2(i - 1) < (n-1)/2$, that is i is $(n+3)/4$.
- The number of BFS traversals is $2 \times (n+3)/4 = (n+3)/2 = \Theta(n)$.

Module 2

Module 2

- We will study how graph **connectivity** algorithms are **developed** over the years and across platforms.
- Sequential/Parallel/...

Graph Connectivity

- The problem of graph connectivity is defined as follows.
- Given an undirected graph $G = (V, E)$, is G connected?
- Connected refers to the property that there exists at least one path between every pair of vertices.
- The traditional algorithms for this problem are to use
- Breadth first search (BFS)
- Depth first search (DFS)
- Both these algorithms take time $O(m+n)$ in the sequential setting.
- In the above, $n := |V|$, and $m := |E|$

Graph Connectivity

- BFS and DFS require all edges to be considered.
- It is natural to say that every edge is important in answering the connectivity question.
- Example: the dumbbell graph.
- But what about real-world networks?

Graph Connectivity

- An important lemma from random graphs is a useful practical aid.
- A random graph is a graph G on n vertices where each edge is included in G with a probability of p , independently.
- G is denoted as being a member of $G(n, p)$, the class of Erdos-Reyni random graphs.

Graph Connectivity

- Theorem: A graph G in $G(n,p)$ is connected with high probability provided $p \geq \ln n/n$.
- Proof: A graph that is not connected misses too many edges.
- Each such edge is missed with a probability of $(1-p)$ independently.

Recent Papers Using the Above Idea

- Several domains of algorithms and theory use the above idea in designing efficient (randomized) connectivity tests.
- Parallel Computing:
- Sutton et al., IPDPS 2018

Towards 2-Connectivity

- An undirected graph $G = (V, E)$ is biconnected iff every pair of vertices u and v in G have at least two vertex disjoint paths in G .
- A biconnected component of G is a maximal subgraph of G that is biconnected.

Graph Biconnectivity

- A standard algorithm for this problem in the sequential computing model is the algorithm of Tarjan.

Graph Biconnectivity

- The algorithm of Tarjan is inherently sequential.
- The first parallel algorithm is the algorithm of Tarjan and Vishkin.
- This algorithm converts the problem of graph biconnectivity to that of connectivity in an auxiliary graph.

The Algorithm of Bader and Cong

- Bader and Cong suggest modifications to the algorithm of Tarjan and Vishkin to improve the parallel performance.

